

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	K.MAHESH BABU	Reg. No.:	192525122
Section / Batch:	CSA1214/2025	Date:	25/08/26

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling-DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time monitoring system receives data from multiple sensors with different sampling frequencies and response-time requirements. Develop a pseudocode for an interrupt scheduling mechanism that assigns suitable interrupt priorities to the sensors based on their sampling rate, urgency, and data-processing requirements. The algorithm should manage simultaneous interrupt requests and ensure that time-critical sensor data is serviced without causing excessive CPU overhead.
2. A high-speed data acquisition system must transfer continuous streams of data between an I/O device and memory while preventing data loss when the incoming data rate exceeds the processing speed of the CPU. Write a pseudocode for a double-buffered DMA mechanism that alternates between two memory buffers during data transfer. The algorithm should detect buffer completion, switch buffers, generate appropriate interrupts, and handle buffer overflow conditions.
3. A computer system contains several I/O devices with different data transfer sizes and timing requirements. Design a pseudocode for an I/O scheduling algorithm that determines the order in which pending I/O requests should be serviced. The algorithm should consider device priority, request size, waiting time, latency requirements, and available bandwidth to improve overall I/O performance while preventing starvation.
4. A high-performance embedded system uses multiple DMA-capable devices that may request memory access at the same time. Develop a pseudocode for a DMA channel arbitration mechanism that allocates memory bus access among competing DMA requests. The algorithm should consider request priority, transfer size, urgency, and fairness, while ensuring that high-priority transfers are completed within their required time constraints.
5. A real-time system must communicate simultaneously with several devices that have different communication speeds and reliability requirements. Write a pseudocode for an adaptive I/O management mechanism that monitors device performance and dynamically changes the communication method or transfer parameters when congestion, delays, or errors are detected. The algorithm should monitor transfer status, detect performance degradation, adjust resource allocation, and maintain reliable communication with minimal CPU overhead.

I K.MAHESH BABU(192525122) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.MAHESH BABU(192525122)_

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator

Faculty Incharge

Q1. Interrupt Scheduling Mechanism for Multiple Sensors

Problem Understanding

The system receives data from multiple sensors with different:

- Sampling frequencies
- Urgency levels
- Response-time requirements
- Data-processing requirements

The algorithm must:

1. Assign suitable priorities.
2. Handle simultaneous interrupts.
3. Service time-critical sensors first.
4. Prevent excessive CPU overhead.

Priority Calculation

A sensor priority can be determined using:

Higher score → Higher priority.

Pseudocode

ALGORITHM PriorityBasedInterruptScheduler

INPUT:

SensorList

For each sensor:

SamplingRate

Urgency

ResponseTime

DataProcessingRequirement

OUTPUT:

OrderedInterruptQueue

ServicedSensor

BEGIN

FOR each Sensor in SensorList DO

PriorityScore ← 0

IF Urgency = "CRITICAL" THEN

PriorityScore ← PriorityScore + 50

ELSE IF Urgency = "HIGH" THEN

PriorityScore ← PriorityScore + 30

ELSE

PriorityScore ← PriorityScore + 10

END IF

IF SamplingRate >= HIGH_RATE_LIMIT THEN

```

    PriorityScore ← PriorityScore + 30
ELSE IF SamplingRate >= MEDIUM_RATE_LIMIT THEN
    PriorityScore ← PriorityScore + 20
ELSE
    PriorityScore ← PriorityScore + 10
END IF

IF ResponseTime <= CRITICAL_RESPONSE_TIME THEN
    PriorityScore ← PriorityScore + 40
ELSE IF ResponseTime <= NORMAL_RESPONSE_TIME THEN
    PriorityScore ← PriorityScore + 20
END IF

IF DataProcessingRequirement = "HIGH" THEN
    PriorityScore ← PriorityScore + 20
END IF

```

```

Sensor.Priority ← PriorityScore

```

```

END FOR

```

```

WHILE SystemRunning = TRUE DO

```

```

    PendingInterrupts ← GetPendingInterrupts()

```

```

    IF PendingInterrupts is NOT EMPTY THEN

```

```

        Sort PendingInterrupts
        by PriorityScore in DESCENDING order

```

```

        FOR each Interrupt in PendingInterrupts DO

```

```

            IF Interrupt.Sensor.Urgency = "CRITICAL" THEN
                ServiceInterruptImmediately(Interrupt)

```

```

            ELSE
                QueueInterrupt(Interrupt)
            END IF

```

```

        END FOR

```

```

        WHILE InterruptQueue is NOT EMPTY DO

```

```

            CurrentInterrupt ← RemoveHighestPriority(InterruptQueue)

```

```

            ServiceInterrupt(CurrentInterrupt)

```

```

        END WHILE

```

```

ELSE

```

```
CPU ← IDLE
END IF
```

```
END WHILE
```

```
END
```

Why this algorithm is suitable

- Critical sensors receive the highest priority.
- Higher sampling rates increase priority.
- Response-time requirements are considered.
- Simultaneous interrupts are sorted by priority.
- CPU remains idle when no interrupt is pending.
- Non-critical interrupts are queued instead of unnecessarily interrupting the CPU.

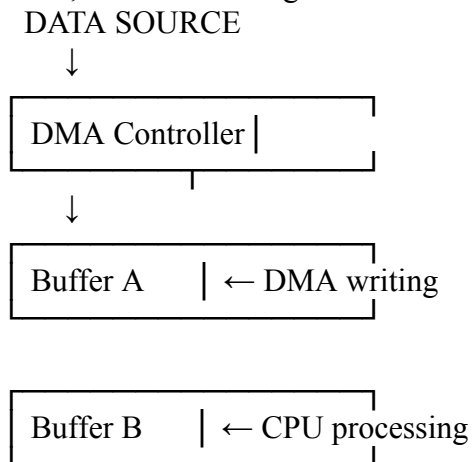
Conclusion

The algorithm provides priority-based and response-sensitive interrupt scheduling. It ensures that time-critical sensor data is processed first while reducing unnecessary CPU overhead.

Q2. Double-Buffered DMA Mechanism

Problem Understanding

A high-speed data acquisition system continuously receives data. If incoming data is faster than CPU processing, a single buffer may become full before the CPU finishes processing it. Therefore, double buffering is used.



When Buffer A becomes full, DMA switches to Buffer B while the CPU processes Buffer A.

Pseudocode

ALGORITHM DoubleBufferedDMA

INPUT:

InputDevice
BufferA
BufferB
BufferSize

OUTPUT:

ProcessedData
OverflowStatus

BEGIN

CurrentDMABuffer ← BufferA
CurrentCPUBuffer ← BufferB

ConfigureDMA(InputDevice, CurrentDMABuffer, BufferSize)

StartDMA()

WHILE SystemRunning = TRUE DO

IF DMA_Transfer_Complete() = TRUE THEN

CompletedBuffer ← CurrentDMABuffer

IF CurrentDMABuffer = BufferA THEN

CurrentDMABuffer ← BufferB

ELSE

CurrentDMABuffer ← BufferA

END IF

IF CurrentDMABuffer.Status = "FREE" THEN

ConfigureDMA(
InputDevice,
CurrentDMABuffer,
BufferSize
)

StartDMA()

ELSE

OverflowStatus ← TRUE

StopDMA()

Report("BUFFER OVERFLOW")

HandleOverflow()

IF FreeBufferAvailable() THEN

CurrentDMABuffer ← GetFreeBuffer()

ConfigureDMA(
InputDevice,
CurrentDMABuffer,

```
        BufferSize
    )
    StartDMA()
ELSE
    WaitForBuffer()
END IF
```

```
END IF
```

```
MarkStatus(CompletedBuffer, "READY")
```

```
GenerateDMAInterrupt()
```

```
END IF
```

```
IF BufferReadyForCPU() = TRUE THEN
```

```
    CurrentCPUBuffer ← GetReadyBuffer()
```

```
    ProcessBuffer(CurrentCPUBuffer)
```

```
    MarkStatus(CurrentCPUBuffer, "FREE")
```

```
END IF
```

```
END WHILE
```

```
END
```

Working

Initially:

Buffer A → DMA

Buffer B → CPU

After Buffer A becomes full:

Buffer A → CPU

Buffer B → DMA

Then:

Buffer B → CPU

Buffer A → DMA

This continues alternately.

Overflow Handling

If both buffers are occupied and DMA has nowhere to write:

DMA Buffer Full

↓

Check Other Buffer

↓

Other Buffer Busy?

↓
BUFFER OVERFLOW

The system then attempts to free or allocate another buffer.

Conclusion

Double-buffered DMA allows data acquisition and data processing to occur simultaneously. It reduces CPU involvement and minimizes data loss when continuous high-speed data is received.

Q3. I/O Scheduling Algorithm

Problem Understanding

The system contains multiple I/O devices with different:

- Priorities
- Request sizes
- Waiting times
- Latency requirements
- Available bandwidth

The algorithm must select the best request while preventing starvation.

Scheduling Score

A suitable score is:

Higher score → serviced first.

Pseudocode

ALGORITHM PriorityAwareIOScheduler

INPUT:

PendingIORequests

FOR each Request:

DevicePriority

RequestSize

WaitingTime

LatencyRequirement

AvailableBandwidth

OUTPUT:

ServiceOrder

BEGIN

WHILE PendingIORequests is NOT EMPTY DO

FOR each Request in PendingIORequests DO

PriorityScore ← 0

PriorityScore ← PriorityScore
+ DevicePriority * PRIORITY_WEIGHT


```

PriorityScore ← PriorityScore
                + WaitingTime * WAIT_WEIGHT

IF LatencyRequirement = "CRITICAL" THEN
    PriorityScore ← PriorityScore + CRITICAL_LATENCY_BONUS

ELSE IF LatencyRequirement = "HIGH" THEN
    PriorityScore ← PriorityScore + HIGH_LATENCY_BONUS
END IF

IF AvailableBandwidth >= RequestSize THEN
    PriorityScore ← PriorityScore + BANDWIDTH_BONUS
END IF

IF WaitingTime > STARVATION_LIMIT THEN
    PriorityScore ← PriorityScore + AGING_BONUS
END IF

Request.Score ← PriorityScore

END FOR

```

```

SelectedRequest ← Request
                  with HIGHEST Score

```

```

Service(SelectedRequest)

```

```

Remove SelectedRequest
from PendingIORequests

```

```

END WHILE

```

```

END

```

Starvation Prevention

The algorithm uses aging.

If a request waits for a long time:

Waiting Time increases



Priority Score increases



Request moves upward



Request eventually gets serviced

Thus, a low-priority device cannot wait indefinitely.

Example

Suppose:

Request	Priority	Waiting Time	Latency
R1	High	5 ms	Normal
R2	Low	50 ms	Critical
R3	Medium	10 ms	Normal

Although R2 has low device priority, its long waiting time and critical latency requirement can increase its scheduling score.

Therefore, it can be selected before another low-urgency request.

Conclusion

The proposed I/O scheduler combines priority, waiting time, latency requirements, request size, and bandwidth availability. Aging prevents starvation while priority and latency requirements ensure important requests are handled quickly.

Q4. DMA Channel Arbitration Mechanism

Problem Understanding

Multiple DMA devices may request memory access simultaneously.

The arbiter must:

- Consider priority.
- Consider transfer size.
- Consider urgency.
- Maintain fairness.
- Complete high-priority transfers within their deadlines.

Pseudocode

ALGORITHM DMAChannelArbitration

INPUT:

DMARequests

FOR each Request:

Priority
TransferSize
Urgency
ArrivalTime
Deadline

OUTPUT:

SelectedDMARequest

BEGIN

WHILE SystemRunning = TRUE DO

Requests ← GetPendingDMARequests()

```
IF Requests is EMPTY THEN
    Bus ← IDLE
    CONTINUE
END IF
```

```
FOR each Request in Requests DO
```

```
    Score ← 0
```

```
    Score ← Score
        + Priority * PRIORITY_WEIGHT
```

```
    IF Urgency = "CRITICAL" THEN
        Score ← Score + CRITICAL_URGENCY_BONUS
```

```
    ELSE IF Urgency = "HIGH" THEN
        Score ← Score + HIGH_URGENCY_BONUS
    END IF
```

```
    IF CurrentTime >= Request.Deadline THEN
        Score ← Score + DEADLINE_BONUS
    ELSE IF
        TimeRemaining(Request) <= DEADLINE_THRESHOLD THEN

        Score ← Score + URGENT_DEADLINE_BONUS
    END IF
```

```
    IF Request.WaitingTime > STARVATION_LIMIT THEN
        Score ← Score + AGING_BONUS
    END IF
```

```
    IF Request.TransferSize <= SMALL_TRANSFER_LIMIT THEN
        Score ← Score + QUICK_TRANSFER_BONUS
    END IF
```

```
    Request.Score ← Score
```

```
END FOR
```

```
SelectedRequest ←
    Request with HIGHEST Score
```

```
GrantBus(SelectedRequest)
```

```

TransferData(
    SelectedRequest,
    SelectedRequest.TransferSize
)

ReleaseBus(SelectedRequest)

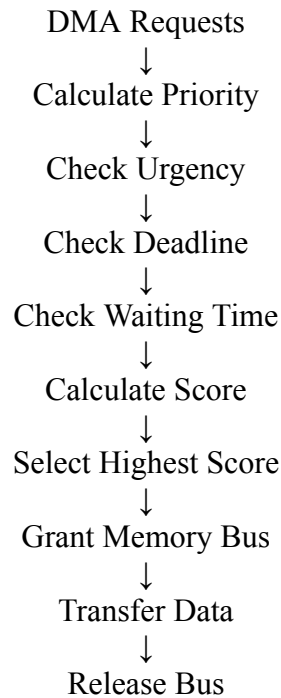
MarkCompleted(SelectedRequest)

END WHILE

END

```

Arbitration Process



Fairness:

If a low-priority DMA request waits for too long:
then:

Therefore, the request eventually receives the bus.

Critical Transfer

If a DMA request has an approaching deadline:

Deadline approaching

↓
Urgency increases

↓
Priority Score increases

↓
DMA request selected

Conclusion

The proposed DMA arbitration mechanism balances priority, urgency, transfer size, deadlines, and fairness.

High-priority transfers receive rapid access to the memory bus, while aging prevents low-priority requests from suffering starvation.

Q5. Adaptive I/O Management Mechanism

Problem Understanding

The real-time system communicates with multiple devices having different:

- Communication speeds
- Reliability requirements
- Transfer rates

The system must dynamically adapt when:

- Congestion occurs
- Delays increase
- Errors occur
- Performance decreases

The paper specifically requires monitoring performance and dynamically changing communication methods or transfer parameters.

Adaptive Strategy

The system continuously monitors:

- Transfer time
- Error rate
- Queue length
- Bandwidth utilization
- Timeout count

Depending on the results, it changes:

- Transfer size
- DMA usage
- Polling interval
- Interrupt frequency
- Resource allocation

Pseudocode

ALGORITHM AdaptiveIOManagement

INPUT:

DeviceList

FOR each Device:

CommunicationMethod

TransferSize

Timeout

RetryLimit

Priority

OUTPUT:

UpdatedIOConfiguration

BEGIN

WHILE SystemRunning = TRUE DO

FOR each Device in DeviceList DO

Status $\leftarrow$ MonitorDevice(Device)

TransferRate $\leftarrow$ CalculateTransferRate(Device)

ErrorRate $\leftarrow$ CalculateErrorRate(Device)

Delay $\leftarrow$ CalculateAverageDelay(Device)

QueueLength $\leftarrow$ GetQueueLength(Device)

BandwidthUsage $\leftarrow$ GetBandwidthUsage(Device)

IF ErrorRate > ERROR_THRESHOLD THEN

ReduceTransferSize(Device)

IncreaseRetryLimit(Device)

IncreaseTimeout(Device)

IF ErrorRate remains HIGH THEN

SwitchToReliableCommunication(Device)

END IF

END IF

IF Delay > DELAY_THRESHOLD THEN

IF Device.Priority = "HIGH" THEN

IncreasePriority(Device)

AllocateMoreBandwidth(Device)

ELSE

ReduceResourceAllocation(Device)

END IF

END IF

IF QueueLength > CONGESTION_THRESHOLD THEN

ReduceTransferSize(Device)

```
IF Device supports DMA THEN
    EnableDMA(Device)
END IF
```

```
    IncreaseBufferSize(Device)
```

```
END IF
```

```
IF TransferRate < MINIMUM_TRANSFER_RATE THEN
```

```
    IF Device supports DMA THEN
        EnableDMA(Device)
    ELSE
        OptimizePollingInterval(Device)
    END IF
```

```
END IF
```

```
IF BandwidthUsage > BANDWIDTH_LIMIT THEN
```

```
    IF Device.Priority = "LOW" THEN
        ReduceBandwidthAllocation(Device)
    ELSE
        MaintainBandwidth(Device)
    END IF
```

```
END IF
```

```
IF ErrorRate < LOW_ERROR_THRESHOLD
    AND Delay < LOW_DELAY_THRESHOLD
    AND QueueLength < LOW_QUEUE_THRESHOLD THEN
```

```
    RestoreNormalTransferParameters(Device)
```

```
END IF
```

```
END FOR
```

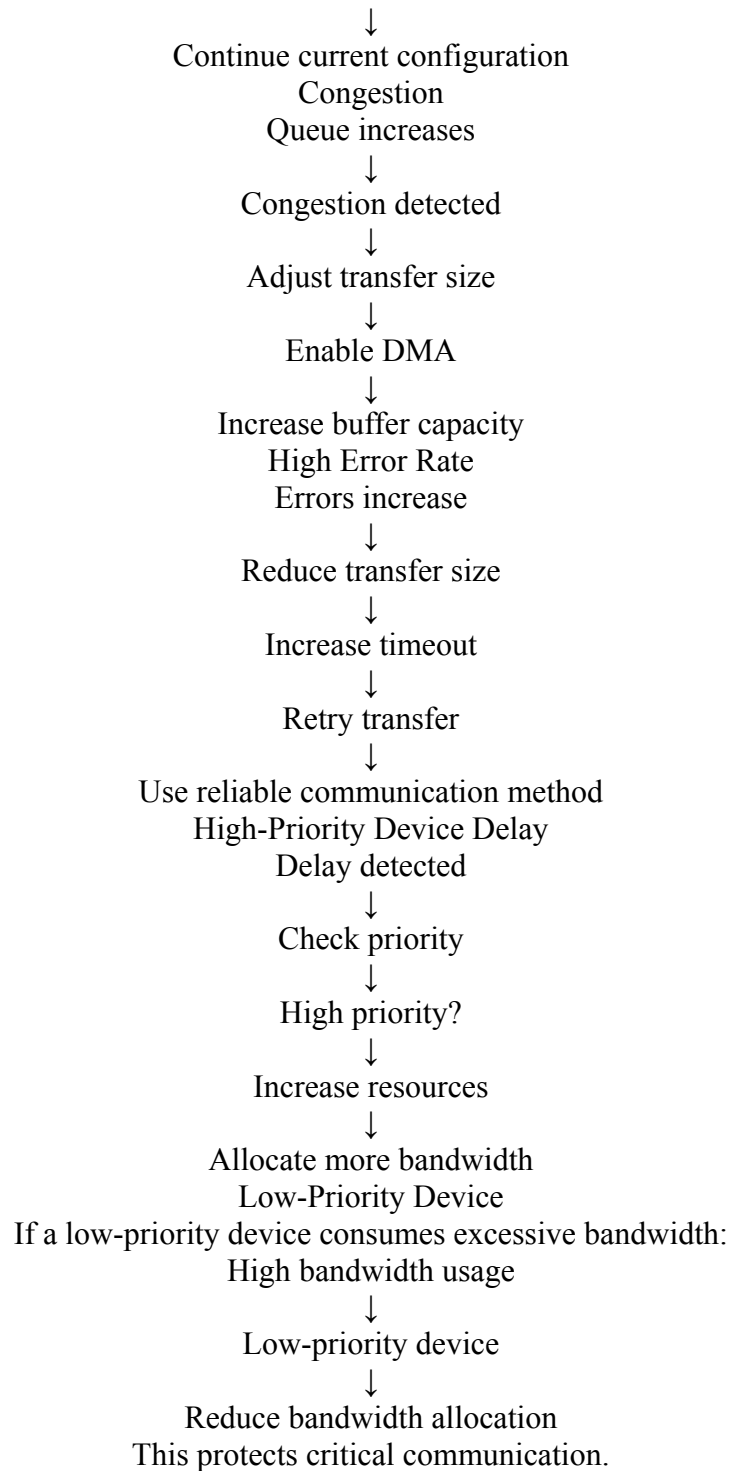
```
Sleep(MONITORING_INTERVAL)
```

```
END WHILE
```

```
END
```

Working of the Algorithm

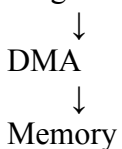
Normal Condition
Normal performance



CPU Overhead Reduction

DMA should be enabled for devices that transfer large continuous blocks.

Large Data Transfer



For small transfers, interrupt-driven or polling methods may be more efficient depending on device behavior. Therefore, the system dynamically chooses the appropriate method.

Conclusion

The adaptive I/O management algorithm continuously monitors transfer rate, errors, delay, queue length, and bandwidth usage. It dynamically changes transfer parameters and resource allocation when performance degradation occurs. DMA is used for high-volume transfers, while priorities protect critical devices. This provides reliable communication with reduced CPU overhead.